M.Tech — Fundamentals of Artificial Intelligence
Mini-Project Report

Student Performance Decision System

using Bayesian Reasoning

Mini-Project Topic No. 7 | Technique: Bayesian Reasoning (Naive Bayes, Bayesian Network, Decision Theory)

Group Members

S.No.	Student Name	Roll Number
1	Soorya S	RA2612044015063
2	Vadivel	RA2612044015100
3		

Date of submission: 31 October 2026

Contents

1. Problem Definition 3
2. AI Problem Formulation 3
3. Selection and Explanation of the AI Technique 4
4. Python Implementation 6
5. Dataset and Test-Case Preparation 8
6. Performance Evaluation 9
7. Comparison with Alternative Approaches 11
8. Results and Discussion 12
9. Demonstration of the Working Prototype 14
10. Conclusion 16

References 16

Abstract

Identifying students at risk of failing early enough to help them is a decision made under uncertainty: the available evidence (attendance, study habits, internal marks, previous grades, assignment completion) is noisy, partly missing, and only probabilistically related to the final result. This project builds a **Student Performance Decision System** that uses **Bayesian reasoning** to compute P(Fail | evidence) and then applies **decision theory** to recommend the intervention with the minimum expected cost. Two Bayesian models are implemented from scratch in Python: a Naive Bayes classifier and a Bayesian network with exact inference by enumeration. They are evaluated on a 2,000-student dataset and compared against an ID3 decision tree, a hand-written rule-based system and a prior-only baseline. The Bayesian network achieved the best cross-validated accuracy (85.9%), the best calibrated probabilities (lowest log loss) and the lowest average decision cost, while also handling missing inputs and supporting diagnostic queries that the alternatives cannot answer.

1. Problem Definition

Academic institutions want to identify students who are likely to fail a course while there is still time to intervene. Teachers usually rely on informal judgement, which is inconsistent, or on fixed rules (e.g. "attendance below 65% means at risk"), which ignore how the different factors combine and give no measure of confidence.

The problem has three characteristics that make it a natural fit for probabilistic AI:

- **Uncertainty** — two students with identical records can get different results; the relationship between evidence and outcome is probabilistic, not deterministic.
- **Incomplete information** — attendance may not be recorded or internal marks not yet published; the system must still reason with whatever is available.
- **Asymmetric consequences** — failing to help a student who goes on to fail is far more costly than giving extra support to a student who would have passed anyway.

Objective. Given any subset of a student's academic indicators, the system must (1) estimate the probability that the student will fail, (2) explain which evidence drove that estimate, and (3) recommend one of three actions — *No action*, *Mentor & monitor*, or *Intensive intervention* — that minimises expected cost.

2. AI Problem Formulation

The task is formulated as **probabilistic inference followed by a decision under uncertainty**. The world is described by six discrete random variables:

Variable	Type	States	Raw measure and cut-offs
StudyHours	Evidence	Low, Medium, High	hours/week (Low <5, Medium 5-12, High >=12)
Attendance	Evidence	Low, Medium, High	% (Low <65, Medium 65-85, High >=85)
PrevGPA	Evidence	Poor, Average, Good	out of 10 (Poor <6, Average 6-8, Good >=8)
InternalMarks	Evidence	Poor, Average, Good	out of 50 (Poor <20, Average 20-35, Good >=35)
Assignments	Evidence	Incomplete, Complete	submitted out of 10 (Incomplete <7, Complete >=7)
Result	Query (hidden)	Pass, Fail	Final course outcome

Table 1. Random variables of the model

Evidence. e = any subset of {StudyHours, Attendance, PrevGPA, InternalMarks, Assignments}; unobserved variables are left unknown rather than guessed.

Query. The posterior distribution P(Result | e).

Actions. A = {No action, Mentor & monitor, Intensive intervention}.

Cost model. Cost(a, s) for each action a and true outcome s (Table 2). Missing a failing student (No action, Fail) carries the largest cost.

Action	Cost if student Passes	Cost if student Fails
No action	0	10
Mentor & monitor	1	4
Intensive intervention	3	1

Table 2. Cost matrix used by the decision layer

Decision rule (maximum expected utility / minimum expected cost):

$$a^* = \operatorname{argmin}_{a \text{ in } A} EC(a), \quad EC(a) = \Sigma_s P(s \mid e) \cdot Cost(a, s)$$

Because EC(a) is linear in p = P(Fail | e), the rule is equivalent to two thresholds: **No action** if $p < 0.143$; **Mentor & monitor** if $0.143 \le p < 0.400$; **Intensive intervention** if $p \ge 0.400$. Note that a student is escalated at p = 0.4, not 0.5, because of the asymmetric costs.

3. Selection and Explanation of the AI Technique

3.1 Why Bayesian reasoning?

- It represents uncertainty explicitly as probabilities, which the decision layer needs in order to compute expected costs — a hard yes/no classifier cannot do this.
- It combines prior knowledge (base pass rate, causal structure) with evidence in a principled way, and updates beliefs incrementally as new evidence arrives.
- Missing evidence is handled exactly by marginalisation, with no ad-hoc imputation.
- It is explainable: each probability can be traced to counts and to the contribution of each piece of evidence.

3.2 Bayes' theorem

$$P(\text{Result} \mid e) = P(e \mid \text{Result}) \cdot P(\text{Result}) / P(e)$$

The **prior** P(Result) is the belief before seeing the student; the **likelihood** P(e | Result) is how typical the evidence is of passing or failing students; the **posterior** P(Result | e) is the updated belief.

Worked example. Let P(Pass) = 0.7, P(Fail) = 0.3, P(Attendance=Low | Pass) = 0.2 and P(Attendance=Low | Fail) = 0.7. Then the unnormalised scores are 0.2 × 0.7 = 0.14 for Pass and 0.7 × 0.3 = 0.21 for Fail; normalising gives P(Fail | Low attendance) = 0.21 / 0.35 = **0.60**. One observation moved the belief from 30% to 60% (this example is checked by a unit test).

3.3 Model A — Naive Bayes

Naive Bayes assumes every feature is conditionally independent of the others given Result:

$$P(\text{Result} \mid e_1, \ldots, e_n) = \alpha \cdot P(\text{Result}) \cdot \Pi_i P(e_i \mid \text{Result})$$

Parameters are learned by counting with Laplace (add-one) smoothing, $P(v \mid c) = (N_{v,c} + 1) / (N_c + |V|)$, so unseen combinations never get probability zero. Products are computed as sums of logarithms for numerical stability. The contribution of each observed feature is its **log-likelihood ratio** $w = \ln[P(v \mid \text{Fail}) / P(v \mid \text{Pass})]$: positive values push towards Fail. This gives a direct explanation of every prediction.

3.4 Model B — Bayesian network

The independence assumption of Naive Bayes is unrealistic: low study hours cause poor internal marks, so treating them as independent evidence double-counts the same signal. The Bayesian network encodes causal domain knowledge instead (Figure 1). The joint distribution factorises along the graph:

$$P(X_1, \ldots, X_n) = \Pi_i P(X_i \mid \text{Parents}(X_i))$$

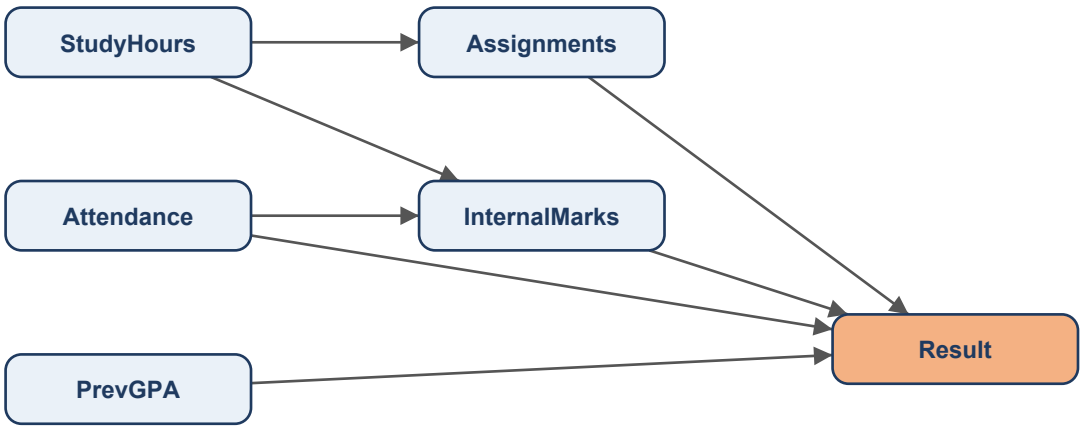


Figure 1. Bayesian network structure (Result shaded)

Learning. Each conditional probability table (CPT) is estimated by counting with Laplace smoothing; a row contributes to a CPT when the node and all its parents are observed.

Inference by enumeration. For a query variable Q, evidence e and hidden variables H:

$$P(Q \mid e) = \alpha \, \Sigma_h \, \Pi_i \, P(x_i \mid \text{parents}(X_i)), \quad \text{summed over all assignments h of H}$$

With six variables of at most three states there are at most $3^5 = 243$ terms per query value, so exact enumeration is fast. The same routine answers **predictive** queries (P(Result | Attendance=Low)), **diagnostic** queries (P(StudyHours | Result=Fail)) and infers unobserved inputs (P(InternalMarks | e)).

3.5 Decision layer

The posterior from the Bayesian network is passed to the decision-theoretic rule of Section 2. The system therefore separates *what is likely* (inference) from *what to do about it* (decision), so the institution can change the costs without retraining any model.

4. Python Implementation

The system is implemented in Python 3 using only the standard library — every algorithm (Naive Bayes, Bayesian network learning and inference, ID3, metrics) is written from scratch. The code is organised as a package `spds` plus three entry-point scripts.

Module	Responsibility
spds/data.py	Variable definitions, discretisation, CSV loading, train/test and k-fold splits
spds/synthetic.py	Synthetic dataset generator (hidden-trait model)
spds/naive_bayes.py	Naive Bayes: learning, posterior, sequential update, evidence weights
spds/bayes_net.py	Bayesian network: topological order, CPT learning, exact enumeration
spds/decision.py	Cost matrix, expected-cost decision, analytic thresholds
spds/baselines.py	ID3 decision tree, rule-based system, prior-only baseline
spds/metrics.py	Accuracy, precision, recall, F1, log loss, Brier score, decision cost
spds/models.py	Bayesian network structure and model factory
run_experiment.py	Training, evaluation, comparison; writes results/report.md
demo.py	Interactive command-line prototype
tests/test_core.py	Unit tests

Table 3. Project modules

4.1 Naive Bayes posterior

```
def predict_proba(self, evidence):
    log_scores = {c: math.log(self.prior[c]) for c in self.classes}
    for f, v in self.observed(evidence):
        for c in self.classes:
            log_scores[c] += math.log(self.likelihood[f][c][v])
    return normalise_log(log_scores)
```

4.2 Bayesian network — CPT learning and exact inference

```
def fit(self, records):
    """Learn every CPT by counting (maximum likelihood + Laplace smoothing).

    A row only counts towards a CPT if the node and all its parents are
    observed in that row (available-case analysis).
    """
    a = self.alpha
    for node in self.order:
        pars = self.parents[node]
        counts = defaultdict(Counter)
        for r in records:
            if r.get(node) is None or any(r.get(p) is None for p in pars):
                continue
            counts[tuple(r[p] for p in pars)][r[node]] += 1
        values = self.states[node]
        table = {}
        for config in itertools.product(*(self.states[p] for p in pars)):
            c = counts[config]
            total = sum(c.values())
            table[config] = {v: (c[v] + a) / (total + a * len(values)) for v in values}
        self.cpt[node] = table
    return self

def joint(self, assignment):
    p = 1.0
    for node in self.order:
        config = tuple(assignment[q] for q in self.parents[node])
        p *= self.cpt[node][config][assignment[node]]
    return p

def query(self, variable, evidence):
    """Exact posterior P(variable | evidence) by enumeration."""
    evidence = {k: v for k, v in evidence.items() if v is not None and k in self.parents}
    if variable in evidence:
        return {v: float(v == evidence[variable]) for v in self.states[variable]}
```

```
        hidden = [n for n in self.order if n != variable and n not in evidence]
        dist = {}
        for value in self.states[variable]:
            total = 0.0
            for combo in itertools.product(*(self.states[h] for h in hidden)):
                assignment = dict(evidence)
                assignment[variable] = value
                assignment.update(zip(hidden, combo))
                total += self.joint(assignment)
            dist[value] = total
        return normalise(dist)
```

4.3 Decision layer

```
def expected_costs(posterior):
    return {a: sum(posterior[s] * COST[a][s] for s in posterior) for a in ACTIONS}

def decide(posterior):
    costs = expected_costs(posterior)
    return min(costs, key=costs.get), costs
```

How to run. `python generate_data.py` creates the dataset; `python run_experiment.py` trains and evaluates all models; `python demo.py` starts the prototype; `python -m unittest discover tests` runs the tests.

5. Dataset and Test-Case Preparation

5.1 Dataset

A dataset of **2000 students** is generated by `spds/synthetic.py` (seed 42, reproducible). Each student has two hidden traits, *motivation* and *ability*, drawn from a standard normal distribution. The observable measures are noisy functions of these traits — e.g. study hours and attendance depend on motivation, previous GPA on ability, internal marks on study hours, attendance and ability — and the final result depends on the observables plus Gaussian noise. The outcome is therefore genuinely uncertain given the evidence, as in real life. About 3% of values are blanked at random to simulate incomplete records. Raw values are discretised using the cut-offs in Table 1.

Variable	State counts	Missing
StudyHours	Low: 485, Medium: 1026, High: 427	62
Attendance	Low: 268, Medium: 1110, High: 557	65
PrevGPA	Poor: 425, Average: 1101, Good: 424	50
InternalMarks	Poor: 405, Average: 1156, Good: 374	65
Assignments	Incomplete: 967, Complete: 979	54
Result	Pass: 1361, Fail: 639	0

Table 4. Dataset summary (fail rate 31.9%)

Splits. A shuffled 70/30 hold-out split (1400 training / 600 test students) and 5-fold cross-validation on the full dataset. The same CSV format can be filled from a real dataset such as the UCI *Student Performance* dataset without changing any code.

5.2 Test cases

Unit tests verify the correctness of the reasoning engine. The key test checks that a Bayesian network with the naive structure (Result as the only parent of every feature) reproduces the Naive Bayes posterior to nine decimal places — two independent implementations agreeing.

Unit test	Result
test_worked_example	OK
test_actions_follow_thresholds	OK
test_numbers_labels_and_blanks	OK
test_bn_with_naive_structure_equals_naive_bayes	OK
test_cpts_are_distributions	OK
test_posteriors_sum_to_one	OK
test_sequential_update_matches_batch	OK

Table 5. Unit test results — 7 run, 0 failed

6. Performance Evaluation

6.1 Metrics

Fail is treated as the positive class because detecting at-risk students is the goal. Both classification quality and probability quality are measured, plus the quality of the final decisions.

Metric	Meaning
Accuracy	Fraction of students classified correctly (Fail if P(Fail) ≥ 0.5)
Precision (Fail)	Of the students flagged as Fail, the fraction who really failed
Recall (Fail)	Of the students who failed, the fraction the system caught — most important
F1	Harmonic mean of precision and recall
Log loss	−mean ln P(true outcome); penalises confident wrong probabilities
Brier score	Mean squared error of P(Fail); probability calibration
Avg decision cost	Mean realised cost of the chosen actions (Table 2); lower is better

Table 6. Evaluation metrics

6.2 Hold-out test set

Model	Accuracy	Precision	Recall	F1	Log loss	Brier	Avg cost
Bayesian Network	0.858	0.864	0.702	0.775	0.349	0.109	1.113
Naive Bayes	0.845	0.786	0.760	0.773	0.368	0.111	1.123
Decision Tree (ID3)	0.847	0.826	0.707	0.762	0.357	0.114	1.185
Rule-based	0.830	0.762	0.740	0.751	2.349	0.170	1.397
Prior only	0.653	0.000	0.000	0.000	0.649	0.228	2.040

Table 7. Results on the 600-student hold-out test set

Bayesian Network	Pred. Fail	Pred. Pass
Actual Fail	146	62
Actual Pass	23	369

Naive Bayes	Pred. Fail	Pred. Pass
Actual Fail	158	50
Actual Pass	43	349

Decision Tree (ID3)	Pred. Fail	Pred. Pass
Actual Fail	147	61
Actual Pass	31	361

Rule-based	Pred. Fail	Pred. Pass
Actual Fail	154	54
Actual Pass	48	344

Table 8. Confusion matrices on the hold-out test set

6.3 5-fold cross-validation

Model	Accuracy	Recall (Fail)	F1	Log loss	Avg cost
Bayesian Network	0.859 ± 0.014	0.691 ± 0.018	0.757 ± 0.021	0.339 ± 0.021	1.100 ± 0.049
Naive Bayes	0.840 ± 0.016	0.733 ± 0.028	0.745 ± 0.024	0.361 ± 0.032	1.106 ± 0.078
Decision Tree (ID3)	0.853 ± 0.016	0.670 ± 0.057	0.743 ± 0.030	0.340 ± 0.021	1.105 ± 0.046
Rule-based	0.832 ± 0.013	0.719 ± 0.013	0.732 ± 0.013	2.321 ± 0.180	1.363 ± 0.099
Prior only	0.680 ± 0.017	0.000 ± 0.000	0.000 ± 0.000	0.627 ± 0.014	1.958 ± 0.052

Table 9. 5-fold cross-validation results (mean ± standard deviation)

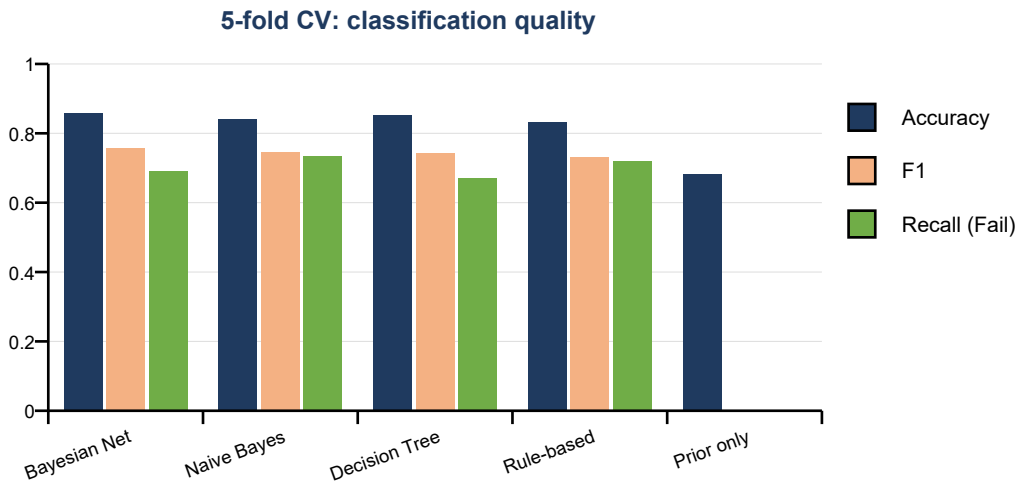


Figure 2. Classification metrics by model

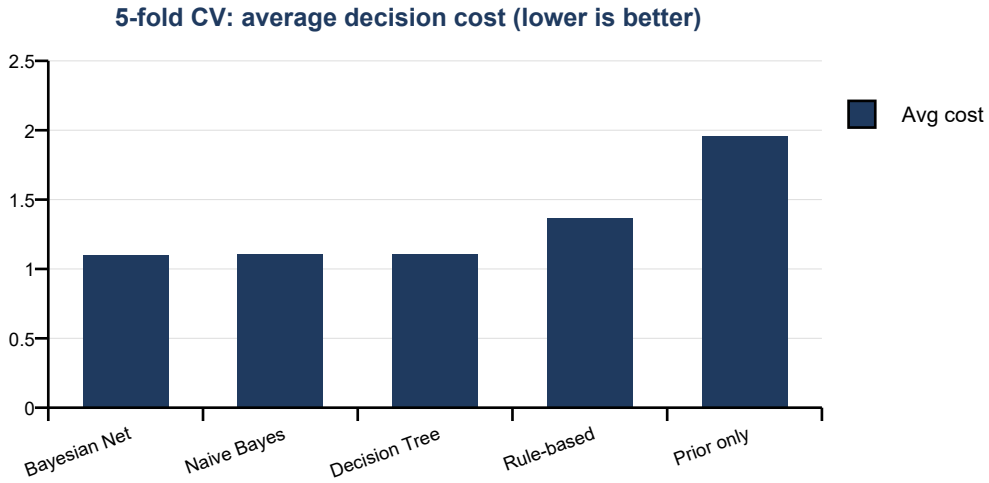


Figure 3. Average decision cost by model

7. Comparison with Alternative Approaches

Three alternative approaches were implemented to put the Bayesian models in context: an **ID3 decision tree** (information-gain splits, maximum depth 4, smoothed leaf probabilities), a **rule-based system** encoding typical teacher heuristics (at risk if attendance is Low, or internal marks are Poor, or previous GPA is Poor and assignments are incomplete), and a **prior-only** baseline that ignores the student.

Capability	Bayesian Net	Naive Bayes	Decision Tree	Rule-based
Calibrated probabilities	Yes (best)	Over-confident	Yes	No (0/1 only)
Handles missing inputs	Exact marginalisation	Skips feature	Stops at node	Partially
Infers unknown inputs	Yes	No	No	No
Diagnostic queries (effect → cause)	Yes	Limited	No	No
Uses domain knowledge	Graph structure	No	No	Hand-written rules
Explainability	CPTs + queries	Evidence weights	Tree paths (large)	Rules
Learns from data	Yes	Yes	Yes	No

Table 10. Qualitative comparison

- **Bayesian network vs Naive Bayes.** The network is more accurate and better calibrated. Naive Bayes has higher recall but more false alarms, and it is over-confident because it double-counts correlated evidence.
- **Bayesian network vs Decision tree.** Accuracy is similar, but the tree is a classifier only: it cannot answer diagnostic queries or infer missing inputs, and its learned structure (tens of leaves) is harder to explain than a six-node network.
- **Bayesian models vs Rule-based.** The rules are reasonable classifiers, but they are certain even when wrong, giving a log loss of 2.32 — about seven times worse. Without probabilities the expected-cost decision cannot be made, and their decision cost is the highest of all approaches except the prior-only baseline.
- **Prior-only.** Shows that the evidence adds real value: decision cost falls by almost half when the student's data is used.

8. Results and Discussion

8.1 Reasoning examples

Case	BN P(Fail)	NB P(Fail)	Decision (BN)
No evidence (prior)	0.311	0.308	Mentor & monitor
Low attendance	0.545	0.657	Intensive intervention
Low attendance + good internal marks	0.393	0.016	Mentor & monitor
High study hours + assignments complete (marks unknown)	0.158	0.037	Mentor & monitor
Weak on every measure	0.955	0.999	Intensive intervention
Mixed profile	0.279	0.305	Mentor & monitor

Table 11. Posterior probabilities and decisions for representative students

- Low attendance alone raises P(Fail) from about 0.31 to above 0.5, enough to trigger intensive intervention.
- Adding *good internal marks* shows the key difference between the models: Naive Bayes collapses to almost 0, treating the marks as independent strong evidence, while the network keeps a moderate risk because attendance also directly affects the final result. The network's answer is the more cautious and realistic one.
- With internal marks unknown, the network infers them from study hours and assignments — P(InternalMarks | e) = Poor 0.04, Average 0.55, Good 0.41, compared with the unconditional Poor 0.19, Average 0.62, Good 0.19.

8.2 Diagnostic reasoning

Reasoning from effect to cause: P(StudyHours) = Low 0.25, Medium 0.52, High 0.23, but P(StudyHours | Result = Fail) = Low 0.38, Medium 0.50, High 0.12. Knowing a student failed makes low study hours about 50% more likely — useful for planning remedial programmes.

8.3 Evidence strength (learned likelihoods)

Feature = value	P(v | Pass)	P(v | Fail)	Weight ln(ratio)
StudyHours = Low	0.140	0.488	+1.25
StudyHours = Medium	0.558	0.450	-0.21
StudyHours = High	0.302	0.061	-1.59
Attendance = Low	0.066	0.283	+1.46
Attendance = Medium	0.544	0.626	+0.14
Attendance = High	0.390	0.092	-1.45
PrevGPA = Poor	0.130	0.395	+1.11
PrevGPA = Average	0.581	0.540	-0.07
PrevGPA = Good	0.289	0.065	-1.49
InternalMarks = Poor	0.026	0.600	+3.15
InternalMarks = Average	0.686	0.398	-0.55
InternalMarks = Good	0.288	0.002	-4.80
Assignments = Incomplete	0.374	0.738	+0.68
Assignments = Complete	0.626	0.262	-0.87

Table 12. Naive Bayes likelihoods; positive weight pushes towards Fail

Poor internal marks are by far the strongest single indicator of failure (likelihood ratio above 20), followed by low attendance, low study hours and poor previous GPA. Good internal marks are almost conclusive evidence of passing.

8.4 Decisions

On the 600-student test set the system recommended: **No action** for 272, **Mentor & monitor** for 129, **Intensive intervention** for 199. The average realised cost fell from 1.96 (no student information) to 1.10 with the Bayesian network.

8.5 Limitations

- The data is synthetic; results should be validated on real institutional data.
- The network structure is designed by hand; it could be learned from data (e.g. hill-climbing with the BIC score).
- CPT learning ignores rows with missing parents; the EM algorithm would use all data.
- Continuous measures are discretised into three states, which loses some information.

9. Demonstration of the Working Prototype

The prototype `demo.py` is a command-line application. A user enters any of the five indicators as numbers (e.g. attendance 58) or category names (e.g. Low); anything left blank is treated as unknown. It can be run interactively (questions are asked one by one) or with arguments. For each student it shows the posterior from both models, the step-by-step Bayesian update, the strength of each piece of evidence, inferred values for unknown inputs, and the expected cost of each action with the recommendation.

Case 1 — at-risk student with incomplete data

```
python demo.py --attendance 58 --study-hours 4 --assignments 5
```

```
=== Evidence ===
  StudyHours     Low
  Attendance     Low
  PrevGPA        (unknown)
  InternalMarks  (unknown)
  Assignments    Incomplete

=== Posterior probability of the final result ===
  Bayesian network  P(Fail) = 0.805  [########################......]   P(Pass) = 0.195
  Naive Bayes       P(Fail) = 0.925  [############################..]   P(Pass) = 0.075

=== Step-by-step Bayesian update (Naive Bayes) ===
  start from the prior                -> P(Fail) = 0.320
  observe StudyHours = Low            -> P(Fail) = 0.595
  observe Attendance = Low            -> P(Fail) = 0.863
  observe Assignments = Incomplete    -> P(Fail) = 0.925

=== Which evidence mattered (log-likelihood ratio, + means towards Fail) ===
  Attendance = Low            P(v|Fail)=0.292 P(v|Pass)=0.068  weight +1.46  towards FAIL
  StudyHours = Low            P(v|Fail)=0.465 P(v|Pass)=0.149  weight +1.14  towards FAIL
  Assignments = Incomplete    P(v|Fail)=0.744 P(v|Pass)=0.380  weight +0.67  towards FAIL

=== Inferred values for unknown inputs (Bayesian network) ===
  PrevGPA       Poor=0.22  Average=0.56  Good=0.22
  InternalMarks Poor=0.67  Average=0.31  Good=0.01

=== Decision (minimum expected cost, using the Bayesian network) ===
  No action               expected cost 8.05
  Mentor & monitor        expected cost 3.41
  Intensive intervention  expected cost 1.39  <== recommended

  RECOMMENDATION: Intensive intervention
```

Previous GPA and internal marks were not supplied; the network inferred that the student most likely has poor internal marks and still recommends intensive intervention.

Case 2 — strong student

```
python demo.py --attendance 88 --study-hours 14 --prev-gpa 8.4 --internal-marks 38 --assignments 9
```

```
=== Evidence ===
  StudyHours     High
  Attendance     High
  PrevGPA        Good
  InternalMarks  Good
  Assignments    Complete

=== Posterior probability of the final result ===
  Bayesian network  P(Fail) = 0.015  [............................]   P(Pass) = 0.985
  Naive Bayes       P(Fail) = 0.000  [............................]   P(Pass) = 1.000

=== Step-by-step Bayesian update (Naive Bayes) ===
  start from the prior                -> P(Fail) = 0.320
  observe StudyHours = High           -> P(Fail) = 0.083
  observe Attendance = High           -> P(Fail) = 0.020
  observe PrevGPA = Good              -> P(Fail) = 0.004
  observe InternalMarks = Good        -> P(Fail) = 0.000
  observe Assignments = Complete      -> P(Fail) = 0.000

=== Which evidence mattered (log-likelihood ratio, + means towards Fail) ===
  InternalMarks = Good        P(v|Fail)=0.002 P(v|Pass)=0.285  weight -5.18  towards PASS
```

```
   StudyHours = High            P(v|Fail)=0.058 P(v|Pass)=0.298  weight -1.65  towards PASS
   PrevGPA = Good               P(v|Fail)=0.060 P(v|Pass)=0.292  weight -1.58  towards PASS
   Attendance = High            P(v|Fail)=0.086 P(v|Pass)=0.382  weight -1.49  towards PASS
   Assignments = Complete       P(v|Fail)=0.256 P(v|Pass)=0.620  weight -0.88  towards PASS

=== Decision (minimum expected cost, using the Bayesian network) ===
   No action                expected cost 0.15  <== recommended
   Mentor & monitor         expected cost 1.05
   Intensive intervention   expected cost 2.97

   RECOMMENDATION: No action
```

10. Conclusion

This project developed a Student Performance Decision System based on Bayesian reasoning. The problem was formulated as probabilistic inference over six discrete random variables followed by a minimum-expected-cost decision over three interventions. Naive Bayes and a Bayesian network with exact inference by enumeration were implemented from scratch in Python, evaluated with classification, calibration and decision-cost metrics, and compared with a decision tree, a rule-based system and a prior-only baseline.

The Bayesian network gave the best overall performance — 85.9% cross-validated accuracy, the lowest log loss and the lowest average decision cost (1.10, against 1.96 without student information). More importantly, it did things the alternatives cannot: it reasoned correctly with missing data, inferred unobserved indicators, answered diagnostic questions, and avoided the over-confidence that Naive Bayes shows when evidence is correlated. Separating inference from the decision lets an institution adjust its intervention costs without retraining.

Future work: validate on real student records; learn the network structure from data; use EM to learn from incomplete records; add continuous (Gaussian) nodes; track students across the semester with a dynamic Bayesian network; and provide a web or GUI front end for teachers.

References

- S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2021 — Chapters 12–13 (Quantifying uncertainty, Probabilistic reasoning) and 15–16 (Making simple decisions).
- J. Pearl, *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*, Morgan Kaufmann, 1988.
- D. Koller and N. Friedman, *Probabilistic Graphical Models: Principles and Techniques*, MIT Press, 2009.
- J. R. Quinlan, "Induction of decision trees," *Machine Learning*, vol. 1, pp. 81–106, 1986.
- P. Cortez and A. Silva, "Using data mining to predict secondary school student performance," Proc. FUBUTEC, 2008 (UCI Student Performance dataset).